



Optimizing GPU Sharing for Container-Based DNN Serving with Multi-Instance GPUs

Xinpeng Wei
Georgia Institute of Technology

Zhichao Li
ByteDance Inc.

Cheng Tan
Northeastern University

ABSTRACT

The trade-off of throughput versus latency is important in serving deep neural networks (DNNs) on GPUs. A hardware feature—Multi-Instance GPU (MIG)—provides an additional dimension to this trade-off. This paper studies the GPU sharing problem for serving DNNs with MIG. We propose a system JORMUNGANDR that allocates MIG-enabled GPUs to optimize a user-defined utility function, and places DNN serving containers on heterogeneous MIG instances. JORMUNGANDR introduces a utility-first search that yields efficient solutions in practice. Additionally, it presents a robust transition protocol that guarantees a seamless switch between two GPU-cluster configurations, minimizing throughput fluctuations. We implement JORMUNGANDR on Kubernetes. Our experiments show that JORMUNGANDR provides near-optimal solutions, using < 6% more GPUs than the optimal solutions.

CCS CONCEPTS

• **Computing methodologies** → **Machine learning**.

KEYWORDS

Machine learning systems, Multi-Instance GPUs

1 INTRODUCTION

Throughput versus latency is a fundamental trade-off in computer systems. People have intensively studied this trade-off in many systems [23, 38, 41]. Serving deep neural networks (also called DNN model inferences) is one of them. Serving DNNs is a process of using trained models to make predictions against unseen inputs. Unlike deep learning training, serving DNN models is often a part of user-facing applications and cares about throughput-latency trade-off.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SYSTOR '24, September 23–24, 2024, Virtual, Israel

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1181-7/24/09...\$15.00

<https://doi.org/10.1145/3688351.3689156>

GPU sharing plays a vital role in this trade-off: using dedicated GPUs provides minimal and stable latencies, but it comes at the cost of low GPU utilization and low throughputs; sharing GPUs increases GPU utilization and throughputs, but with long and unpredictable tail latencies [32].

In the current cloud native environment (e.g., Machine Learning as a Service), DNN services often run within containers [9, 12]. Then, GPU sharing can be framed as a resource allocation problem: how to allocate (shared) GPUs to containers that serve different DNN models. Note that resource allocation is different from, but complementary to, scheduling. Resource allocation systems—for example, HiveD [55], GSLICE [22], Morphling [49]—make strategic, long-term decisions regarding resource configurations for multiple services; whereas DNN serving scheduling systems, like Clipper [21], Clockwork [27], and Reef [29], focus on quick responses to workload fluctuations like request spikes.

In this paper, we study DNN models that can benefit from GPU sharing; for models that already exhaust GPU resources, such as Large Language Models (LLMs), GPU sharing is not advisable and therefore they are not the focus of this paper.

Resource allocation for serving GPU-shared DNN models was easy, for two reasons.

- (1) *Model inference has stable and predictable performance.* This is observed by Gujarati et al. [27], and we confirmed (§8). Serving a DNN model is a computational heavy task and there is little non-determinism. In addition, serving tasks usually solely occupies GPUs without worrying being interrupted by other tasks. Thus, profiling model performance in the same environment is accurate to predict model throughputs and latencies in production.
- (2) *People have few parameters to tune the trade-off.* For a model running on a given hardware and software setup, people often use two approaches to trade off latencies for higher throughputs: batching user requests and using NVIDIA's Multi-Process Service (MPS) [11], which enables running multiple model instances simultaneously on a GPU. Therefore, the search space is small; by deciding the batch size and the number of MPS instances (i.e., (batch size, MPS)), the DNN model's throughput and latencies are determined.

Sharing GPUs has more choices when NVIDIA introduced a hardware feature—Multi-Instance GPUs (MIG) [8]. MIG allows developers to partition one GPU into multiple MIG

instances. Unlike MPS instances, different MIG instances are hardware isolated in both performance and errors. For example, NVIDIA A100 GPUs implement MIG, and one A100 can be partitioned into as many as 7 MIG instances; each instance accounts for 1/7 of the total resources. People can also partition A100 into heterogeneous instances, for example, three instances with 1/7, 2/7, and 4/7 resources. MIG supports *dynamic reconfiguration*—one can partially re-partition a GPU without having to stop other running instances. Section 2 introduces MIG in more detail.

MIG adds a new trade-off dimension to the throughput-latency trade-off, which brings the opportunity to improve the “best trade-off points”—Pareto efficient solutions. Figure 1 describes two Pareto curves for running a model on an A100 GPU, one with MIG and one without MIG. We try all possible configurations and log the throughput and average latency of each configuration. We can see that (a) MIG provides many more points on the Pareto curve, and (b) the Pareto efficient solutions with MIG produce better throughputs than ones without MIG: given similar latencies, the highest-throughput point with MIG surpasses the non-MIG point by 9.5%. Furthermore, the MIG-enabled configurations offer a superior highest throughput (5% higher than non-MIG) owing to the hardware performance isolation of MIG, which mitigates contentions on GPU cache and bandwidth.

However, these improvements come at the cost of complicating the resource allocation problem. In fact, MIG fundamentally changes the complexity of the problem. It turns out that allocating MIG-enabled resources is a new abstract problem that we define as *the Reconfigurable Machine Allocation Problem (RMAP)*. RMAP is an NP-hard problem (§3.3).

Beyond theory, RMAP is also hard in practice. First, MIG increases the search space exponentially, hence simple search approaches no longer work. Consider a simple example of serving one model on one GPU with $MIG=[4/7, 2/7, 1/7]$ (this is short for a GPU partition with a 4/7 instance, a 2/7 instance, and a 1/7 instance). If the number of (batch size, MPS) configurations is N , then it is N^3 with $MIG=[4/7, 2/7, 1/7]$. Moreover, these are heterogeneous MIG instances; thus, one likely requires different configurations for different MIG instances. Also, notice that $MIG=[4/7, 2/7, 1/7]$ is one of 19 MIG partitions [8], and this is just serving a single model on a single GPU. Second, allocating MIG instances follow specific rules and classic resource allocation policies (e.g., allocating CPUs, memory, disk) are not applicable. For example, A100 does not

allow allocating 5/7 or 6/7 instances, and certain combinations are prohibited such as $MIG=[3/7, 3/7, 1/7]$. Section 2 describes these constraints.

In this paper, we introduce JORMUNGANDR, a system that allocates MIG-enabled GPUs for DNN serving containers. Our goal is to help model owners explore the vast GPU configurations and make the wanted throughput-latency trade-off. In particular, users of our system can specify their objectives, for example, having vgg19 throughput ≥ 1000 req/s while keeping 99%-tile latency < 30 ms. Then, JORMUNGANDR automatically partitions GPUs and allocates MIG instances to model containers to satisfy user objectives (called a *deployment*). When containers or user performance objectives get updated, JORMUNGANDR re-plans GPU configurations and issues a robust transition, switching from the old configurations to the new one. By “robust”, we mean that the transition will never violate the performance constraints in users’ objectives (e.g., the throughput of vgg19 should always be ≥ 1000 req/s).

JORMUNGANDR consists of two main components: an *allocator* and a *deployer*. The allocator is responsible for generating and optimizing GPU configurations. We introduce a *utility-first search* algorithm that runs fast and can produce efficient GPU configurations in practice. In our experiments (§8.2), the utility-first search generates near-optimal solutions (using $< 6\%$ more resources than the optimal solutions).

The deployer is in charge of applying the planned configurations to GPUs. When the allocator produces a new deployment to incorporate changes, the deployer need to transition GPU configurations without violating performance requirements. The deployer achieve this by running an *exchange-and-compact* procedure. The exchange-and-compact splits the transition into two phases—an exchange phase to update each model’s MIG instances and a compact phase to repartition GPUs—which detangles GPU partitioning and model placement. The deployer also guarantees that during transitions, model throughputs are always no less than either the old deployment or the new deployment, whichever is smaller.

The contributions of this paper are as follows:

- *A utility-first search to address a new GPU sharing problem (§3, §4)*. We study the problem of allocating MIG-enabled GPUs and define the problem in theoretical terms (§3). In addition, we design a novel algorithm, utility-first search, that runs fast and can produce efficient solutions in practice.
- *A robust transition by exchange-and-compact (§6)*. We design a solution to switch deployments that guarantees to satisfy performance requirements.
- *A built system and experimental evaluation (§6, §7, §8)*. We implement JORMUNGANDR on Kubernetes and address multiple implementation challenges due to deficiency of today’s tooling (§7). Further, we experiment with JORMUNGANDR on an eight A100 GPU cluster. JORMUNGANDR

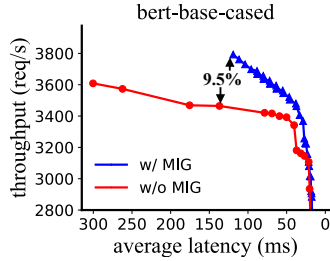


Figure 1: Pareto curves of a DNN model, with and without MIG.

can save 30% of GPUs compared to using A100 disabling MIG. Also, JORMUNGANDR is able to finish deployment transitions on real-world workloads within 1–2 minutes.

As a resource allocator, JORMUNGANDR is applicable to cloud native environments with multiple MIG-enabled GPUs. For example, companies that run on-premises DNN models can use JORMUNGANDR to manage their GPU resources. When deploying DNN models, deployers specify their objectives, then JORMUNGANDR automatically orchestrates containers on the company’s shared GPUs. Another example is cloud Machine Learning as a Service (MLaaS). In this scenario, MLaaS users upload or choose DNN models that run on cloud’s containers. The cloud provider then runs JORMUNGANDR to optimize GPU utilization meanwhile maintaining service QoS.

2 BACKGROUND: GPU SHARING

GPU sharing, also known as GPU multiplexing or spatial sharing [20, 22], indicates the concurrent utilization of a physical GPU by multiple DNN serving processes. In a container-based setup, these processes can be distributed across multiple containers. Next, we introduce the two major GPU sharing methods, Multi-Instance GPU [8] and Multi-Process Service [11].

Multi-Instance GPU (MIG). MIG is a hardware feature that allows users to partition a GPU into multiple instances. Each MIG instance functions as a full-fledged GPU. This paper targets A100 GPUs, which can be partitioned at maximum into 7 instances of the same size. Other MIG-enabled GPUs are NVIDIA A30, H100, and H200.

A100s allow heterogeneous partitions. For example, a GPU can be partitioned into three instances with 1/7, 2/7, and 4/7 of the total resources each. In the rest of the paper, we call an instance with 1/7 of total resources as a 1/7 instance (similarly for instances of other sizes).

MIG is a hardware-based GPU sharing solution: instances have separate streaming multiprocessors, GPU memory, and L1/L2 cache. In addition, instances provide fault and performance isolation by having dedicated on-chip crossbar ports, memory controllers, and DRAM address buses. Essentially, MIG instances are full-fledged GPUs, which are packed in the same “metal box”.

Different from traditional resources, such as CPUs, disks, and memory, MIG’s allocation follows specific rules. Having n units of free resources does not imply being able to allocate an $n/7$ instance. On the one hand, resources can only be grouped into specific sized instances—1/7, 2/7, 3/7, 4/7, 7/7 instances, whereas others (5/7 and 6/7 instances) are not allowed. On the other hand, the existing instances also influence the possible allocations. As an example, for a GPU with two running 3/7 instances, allocating a 1/7 instance is prohibited.

Figure 2 depicts the basic MIG allocation rules. But, there are several exceptions. For example, “3/7 + 3/7” is possible

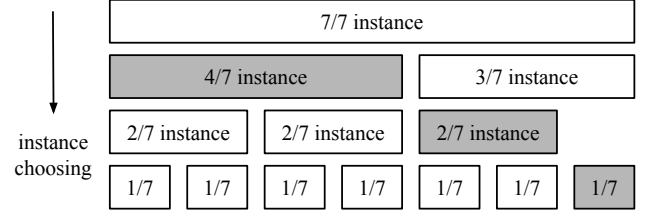


Figure 2: Instance allocation rules, borrowed from NVIDIA’s blog [7, Figure 1]. A legal A100 partition picks instances top-down. If an instance is chosen (for example, the shaded 4/7 instance), then all the instances beneath it are unavailable. As an example, the shaded boxes represents a legal partition with a 4/7, a 2/7, and a 1/7 instance.

in practice but is not shown in the figure. In total, there are 19 legal instance combinations for an A100 GPU (see the full list in NVIDIA’s document [8]).

Multi-Process Service (MPS). NVIDIA MPS [11] serves as a software solution to enable GPU sharing. It allows multiple processes to transparently share a GPU without modifying the applications. With MPS, developers have control over computational resources at software level, allowing them to configure the allocation of computation percentages to individual processes. While MPS provides memory protection by containing software exceptions, the protection is limited: fatal GPU exceptions or errors in the MPS daemon will impact all MPS processes on the same GPU, potentially crashing all MPS processes on the GPU. Furthermore, MPS lacks the capability to set memory usage limits for different MPS processes or isolate performance interference (e.g., contentions on GPU cache and bandwidth).

3 MIG-ENABLED RESOURCE ALLOCATION PROBLEM

Problem statement. In this paper, we assume a container-based environment, in which all DNN serving processes run within containers. Each container will be assigned a GPU (i.e., one MIG instance) and uses MPS to share the GPU across multiple DNN serving processes. These processes will serve end-user requests by running model serving software, such as TensorFlow Serving [40], TorchServe [2], NVIDIA Triton [1]. To reduce the configuration space, we use *homogeneous* MPS processes in a container: all MPS processes are dedicated to serve one DNN model, and they use the same batch size. The problem we study is to allocate GPU resources to containers, such that they satisfy the required service-level objects.

Allocating MIG-enabled resources for serving DNN models is a new problem (formally defined in §3.3). It combines partitioning MIG-enabled GPUs and allocating heterogeneous MIG instances to containers. Moreover, the partitioning and allocating are not isolated but intertwined; one have to solve

the two together to get efficient GPU configurations. Given the problem (partitioning and allocating), our target is to optimize objectives that model deployers specify, for example, maximizing throughputs while keeping tail latencies smaller than a threshold. Model deployers use a utility function to describe their objectives, which we discuss later (§4.2).

As a preview, this problem is computationally challenging and is NP-Complete. Next, we discuss the challenges, from a toy setup to more realistic ones.

3.1 Single GPU, Single Model

This is the simplest case where we serve only one DNN model on one A100 GPU. There are three parameters: (a) MIG (i.e., how GPU is partitioned), (b) MPS (i.e., how many MPS instances in each container), and (c) serving batch sizes. Batch sizes define the maximum requests to serve at the same time, which trade off latency for throughput; namely, the larger the batch size, the higher the throughput and the longer the latency. Because we use homogeneous MPS processes, batches sizes within one container are the same.

Defining GPU configuration. In our problem, we define GPU configuration as a tuple of the above three parameters. For example, $\langle \text{MIG}=[3/7, 3/7], \text{MPS}=[1, 2], \text{batch}=[8, 16] \rangle$ is a GPU configuration, meaning that the GPU allocates two 3/7 instances; the first 3/7 instance container runs one MPS instance with a batch size of 8; the second 3/7 instance container has two MPS instances, both of which use 16 as the batch size. For each GPU configuration, we can profile model performance and record its throughput and a series of latencies (e.g., average, mean, 90%tile, 95%tile, 99%tile latencies).

Challenge 1: the number of GPU configurations is enormous and searching for a desired GPU configuration is non-trivial. For example, assume we limit MPS options to $\{1, 2, 4\}$ and batch sizes in $\{1, 2, 4, 8, 16, 32\}$ (we discuss the candidate batch sizes in §10). Then, for a $\text{MIG}=[2/7, 2/7, 1/7, 1/7, 1/7]$, the number of possible GPU configurations is $(3 * 6)^5 \approx 1.9\text{Million}$. The “3” and “6” are possible MPS options and batch sizes, respectively; the “5” is the number of MIG instances. Notice that $\text{MIG}=[2/7, 2/7, 1/7, 1/7, 1/7]$ is one of 19 MIG partitions.

3.2 Multiple GPUs, Multiple Models

Next, we consider serving multiple models on a single GPU, which we will later extend to multiple GPUs. The first difficulty is that simple metrics (e.g., throughput) are uncomparable across models. For example, a 1/7 instance can serve vgg19 with 175 requests per second, while it can serve 400 requests for bert-base-cased (both with $\text{MPS}=1$ and batch size of 8). The 175 req/s and 400 req/s are not comparable because they are from different models.

Defining in-demand throughput. To compare across models, we define a normalized throughput, called *in-demand throughput*. In-demand throughput for a container (say with 1/7 instance) is the throughput produced by the instance over the demanded throughput. For example, if users require 2K req/s for vgg19 and 8K req/s for bert-base-cased, then we say 1/7 instance can provide 0.0875 ($=175/2K$) in-demand throughput for vgg19 or 0.05 ($=400/8K$) for bert-base-cased. So, we prefer to run vgg19 on 1/7 instance because $0.0875 > 0.05$.

Challenge 2: efficient GPU sharing across multiple models requires accommodating different models’ performance characteristics. For example, some model prefer small MIG instances, while others prefer larger ones. For example, considering two GPU configurations (assuming $\text{MPS}=1$ and batch size of 8): (a) Config-1 allocates $\text{MIG}=[1/7, 2/7, 4/7]$ with the 1/7 instance running vgg19 and others running bert-base-cased; (b) Config-2 allocates $\text{MIG}=[1/7, 1/7, 1/7, 1/7, 3/7]$ with all 1/7 instances running bert-base-cased and the 3/7 instance running vgg19. It turns out that Config-1 has a total in-demand throughput 0.29 (0.09 and 0.2 for vgg19 and bert-base-cased, respectively), whereas Config-2 produces 0.48 (0.28 and 0.2) that is strictly better than Config-1.

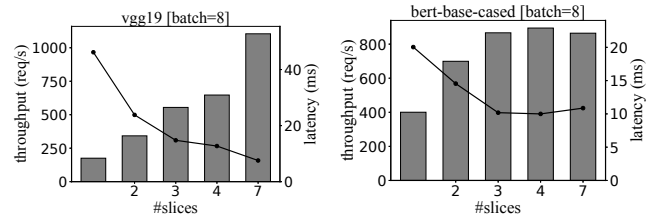


Figure 3: Model throughputs and latencies of instances of different sizes. The x-axis is the instance sizes, where “1” represents a 1/7 instance, and so forth. Bars represent throughputs of the corresponding instances; the line represents the 99%tile latencies.

The reason why some GPU configurations are better than others is that models have different characteristics. Figure 3 shows throughputs and latencies of different sized MIG instances for vgg19 and bert-base-cased. We see that bert prefers smaller instances because it cannot saturate computation resources of larger instances like 4/7 instances, whereas vgg19 prefers larger instances. These different performance characteristics are why Config-2 works better than Config-1.

Defining deployment. Finally, we consider the full-fledged problem of allocating multiple GPUs for multiple models. We define the allocation solution as a *deployment*: a deployment is a set of GPU configurations (§3.1), each of which is a tuple of $\langle \text{MIG}, \text{MPS}, \text{batch sizes} \rangle$. The multi-GPU, multi-model problem combines the mentioned two challenges, leading to an exponentially growing search space of deployments.

3.3 The Reconfigurable Machine Allocation Problem

We describe the MIG-enabled resource allocation problem in this section, which we abstract as *the Reconfigurable Machine Allocation Problem* (short as *RMAP*). RMAP is a new abstract problem and it is NP-hard.

RMAP is defined as follows. There are a set of jobs and a set of machines; each machine can process one job at a time; different machines have different processing time for different jobs. Machines are *reconfigurable*: a set of machines can be rearranged to a different set of machines, under some pre-defined *reconfiguration rules*. And the goal is to find the machine configurations and job assignments that optimize some given objective, for example, maximizing throughputs while satisfying job deadlines.

The problem that we focus on (namely, serving DNNs with MIG) is an instance of RMAP. In our problem, GPU instances are the machines; DNN model inferences are the jobs; different models have different performance on instances of different sizes (DNNs have different performance, §3.1). A set of instances in one GPU can be re-partitioned to another set of instances (a reconfiguration). The reconfiguration rules are the MIG hardware restrictions (§2). Our goal is to find the most efficient GPU partitions and service assignments that optimize the user-defined utility function (defined in §4.2).

A new abstract problem. The Reconfigurable Machine Allocation Problem is a new abstract problem. Resource allocation and scheduling is a broad topic that has been intensively studied. There are many abstract problems that consider reconfiguration in several forms [15, 25, 33–35, 37, 53], but none of them fully captures the characteristics of MIG. We compare RMAP with the most relevant problems in section 9.

An NP-hard problem. Serving DNNs with MIG is an NP-hard problem because another NP-hard problem, *Cutting Stock Problem* [4], reduces to it. The cutting stock problem studies how to cut standard-sized paper rolls into certain numbers of specified-sized pieces while minimizing wasted material. This problem reduces to our problem by treating the paper rolls as GPUs, specified-sized pieces as instances of different sizes, and the required piece numbers as demands. If one can find the minimum number of GPUs for our problem, we have the minimum paper rolls for the Cutting Stock Problem, which minimizes the wasted materials.

4 SEARCHING EFFICIENT GPU CONFIGURATIONS

In this section, we introduce an algorithm to discover efficient solutions for allocating MIG-enabled resources, called *utility-first search*. We first introduce a basic version (§4.1) that maximizes throughputs while ensuring tail latencies are

smaller than a given threshold. Then, we generalize the utility-first search to other objectives (§4.2).

4.1 Basic utility-first search

Utility function. The core idea of the utility-first search is to use a *utility function* that enables comparing among different configuration candidates, and meanwhile optimizes the user-defined objective. For a GPU configuration, its utility function is defined as follows:

$$\text{UTILITY}(gconf, demands, weights) = \sum_{model \in gconf} \frac{gconf[model].throughput}{demands[model]} * weights[model]$$

where

- *gconf* represents the performance of a GPU configuration, encompassing throughput and latency for each model deployed on this GPU;
- *demands* contain required throughput for each model defined by users;
- *weights* consist of numbers $\in [0, 1]$, representing the weights of each model, described below.

In our algorithm, the weights are defined as percentage of unsatisfied required throughputs. For example, if vgg19 requires 2K req/s in total and 800 req/s have already been satisfied, then the $weights[vgg19] = (2K - 800)/2K = 0.6$. In other words, the utility of a GPU configuration is the sum of weighted in-demand throughputs (§3.1) of all served models.

This utility function captures three desired properties. First, our objective is maximizing throughputs; for the same model, the utility function is monotonically increasing according to throughputs. Second, this utility function uses in-demand throughput (§3.1) to compare across models. Finally, the utility function prioritizes models that are less fulfilled, because the weight design prioritizes models with large demands (hence large weights), which are the ones with high resource requirements, indicating the need for careful allocations.

Profiling model performance. As mentioned (§1), serving DNN models have stable performance, an observation made by Gujarati et al. [27]. We confirm that given the same GPU configurations, model throughputs vary $< 3\%$ across experiments. Therefore, for a model, we can profile its throughput and latencies—including average, mean, 90%tile, 95%tile, 99%tile latencies—and reuse them in utility-first search. During profiling, we do an exhaustive search on all container configurations: (a) five MIG instances (1/7, 2/7, 3/7, 4/7, and 7/7 instances), (b) three options for container's MPS processes (1, 2, and 4 MPS processes), and (c) six batch sizes (1, 2, 4, 8, 16, and 32). In total, for each model, we conduct 90 ($= 5 \times 3 \times 6$) profiling experiments, which takes tens of minutes to finish.

Basic algorithm. Figure 4 depicts the algorithm of utility-first search. The basic algorithm is as follows. The algorithm iteratively make decision for one GPU at a time. For each GPU, the algorithm calculates utilities of all GPU configurations from a set of candidates; the candidate set is pre-computed and one GPU serves three models at most. Then, the algorithm chooses the GPU configuration with the maximum utility for this GPU. Utility-first search repeats this process until all models' demands are met.

In particular, the utility-first search begins by initializing a set of possible GPU configurations (*conf_set*), each containing up to three different models that meet required latency thresholds. If a model's configuration—for example, $\langle \text{MIG}=3/7, \text{MPS}=2, \text{batch}=16 \rangle$ —fails the latency requirement, we will remove it from *conf_set*. The algorithm then creates an empty deployment list and maps model demands to their required throughput. Weights are assigned to each model (initially 1), and the algorithm iteratively configure each GPU. For each GPU, the algorithm computes the utility of every configuration in *conf_set* using the utility function (§4.1), which sums the weighted throughput of models in the configuration. The configuration with the maximum utility is then selected and added to the deployment. If the algorithm satisfies all demands or runs out of GPUs, it terminates.

Near-optimal deployment in practice. Utility-first search performs well in practice. It can produce near-optimal solutions in our experiments (§8). Here are some insights why it works. First, though simple, the weighted utilities capture the essence of the optimization problem: it prioritizes the configurations that greatly satisfy user demands, meanwhile balances the long-term allocations by using weights. Second, to manage the large search space, we limit deployment to a maximum of three different models per GPU. This limit is based on empirical observations: serving at most two different models sacrifices performance, while serving more than three causes significant slowdowns, often resulting in the algorithm timing out (10 minutes) when more than a dozen models are involved. Finally, the utility-first search is a greedy algorithm that runs fast but avoids the common pitfall of greedy algorithms, where an early suboptimal choice can lead to irreparable outcomes. This advantage arises because GPUs are interchangeable; a poor decision for the first GPU has the same impact as one for the last. This characteristic of the problem—DNN inferences—happens to favor the algorithm.

Optimizations. We apply multiple optimizations to the utility-first search, both for improving deployment efficiency and algorithm runtime. First, we conduct a “last-mile mixing” for better GPU configuration efficiency. The idea is that when one model, such as vgg19, is close to reaching its required throughput, we temporarily increase the number of different models served on a single GPU by one. Since vgg19 will use

```

1: procedure UTILITYFIRSTSEARCH(numGPU)
2:   conf_set  $\leftarrow$  a set of GPU configurations \
3:     that contains  $\leq 3$  models and have required tail latencies
4:   deployment  $\leftarrow$  an empty list of length numGPU
5:   demands  $\leftarrow$  a map from model to its required throughput
6:
7:   weights  $\leftarrow$  {model1 : 1, ...} // assign 1 for model weights
8:   for i in [1, ..., numGPU]:
9:     utils  $\leftarrow$  an empty map
10:    for conf  $\in$  conf_set: // calc utilities of all configs
11:      utils[conf]  $\leftarrow$  UTILITY(conf, demands, weights) // ln 21
12:    best_conf  $\leftarrow$  PICKCONFIG(utils) // line 27
13:
14:    deployment[i] = best_conf
15:    for m  $\in$  best_conf: // update model weights ( $\geq 0$ )
16:      weights[m]  $\leftarrow$   $\min(\frac{\text{best\_conf}[m].\text{throughput}}{\text{demands}[m]}, \text{weights}[m])$ 
17:
18:    if sum(weights) = 0:
19:      return deployment // find a valid solution
20:
21:  return None // run out of GPUs
22:
23: procedure UTILITY(conf, demands, weights)
24:   util  $\leftarrow$  0
25:   for model  $\in$  conf:
26:     util  $\leftarrow$   $\frac{\text{conf}[\text{model}].\text{throughput}}{\text{demands}[\text{model}]} * \text{weights}[\text{model}]$  // §4.1
27:   return util
28:
29: procedure PICKCONFIG(utils)
30:   conf  $\leftarrow$  the config that has the max utility in utils
31:   return conf

```

Figure 4: JORMUNGANDR's utility-first search.

few resources at this stage, serving additional models is likely to yield more efficient GPU configurations.

Second, utility-first search algorithm has some computational heavy loops, for example, looping all GPU configurations to calculate their utilities (line 10–11, Figure 4). We can vectorize these computation by organizing GPU configurations in a multi-dimensional array and computing their utilities in batch. JORMUNGANDR's implement is in Python (§7), so we use NumPy which leverages hardware SIMD (Single Instruction Multiple Data) instructions.

Third, we cache GPU configurations to avoid repeated computation. This cache is on disk and shared across different executions. The cache is available for the algorithms that share the same problem setup—the same DNN models, demands, and algorithm parameters. Caching significantly accelerate solving large problems.

4.2 Algorithm variants

Considering continuous allocations. The core algorithm (Figure 4) does a one-shot allocation: it allocates GPU configurations assuming all GPUs are empty. However, as we will show later (§5), JORMUNGANDR works continuously, and

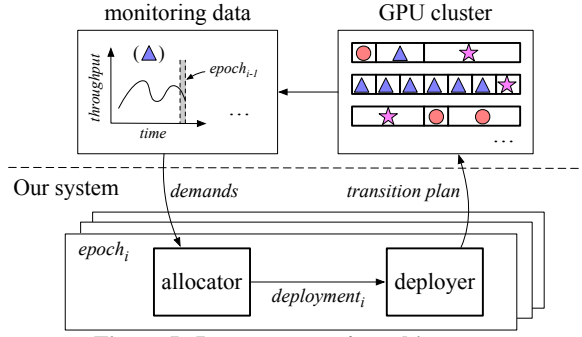


Figure 5: JORMUNGANDR's architecture.

GPUs are usually not empty. In practice, we might want to sacrifice some GPU efficiency to minimize changes between adjacent deployments, as repartitioning GPUs and migrating models can be costly.

To incorporate this concern, JORMUNGANDR tunes its GPU configuration selecting policy. The default policy is to choose the configuration that has the largest utility (line 27, Figure 4). Instead, JORMUNGANDR considers the top- K ($K = 10$ by default) GPU configurations and picks the one that is similar to some existing configurations in the current deployment, so that some containers can run untouched between two deployments. Our similarity function is simple—we count the same MIG instances between two GPU partitions; the higher, the better.

User-defined utility functions. By customizing the utility function, the utility-first search can optimize objectives beyond maximizing throughput. For instance, one could optimize for latency by designing a utility function that favors shorter mean and tail latencies while penalizing longer ones. Similarly, a utility function can be crafted to balance throughput and latency by combining throughput-focused and latency-focused utility functions.

5 SYSTEM OVERVIEW

With utility-first search, we design and implement JORMUNGANDR, a cloud-native resource allocation system that automatically orchestrates DNN serving containers on a cluster of MIG-enabled GPUs. This section introduces JORMUNGANDR's architecture and how JORMUNGANDR works.

Architecture. Figure 5 depicts JORMUNGANDR's architecture. JORMUNGANDR has two main components: an allocator and a deployer. At a high level, the allocator calculates an efficient deployment for the required demands. The demands include required throughputs from end-users' needs and required tail latencies that model owners specify. Later, the deployer implements this deployment on GPUs without interrupting end users. The allocator runs utility-first search (§4), and we will introduce the deployer in section 6.

Workflow. When deployed on a MIG-enabled GPU cluster, JORMUNGANDR initializes its model performance data by profiling models across various GPU configurations, specifically profiling each model in all container setups (§4.1). To expedite the profiling process, JORMUNGANDR utilizes MIG to co-locate the profiling of different models on the same GPU. This initialization typically takes several hours to complete for tens of models. When adding a new model, JORMUNGANDR performs this initialization for the model before it can be served.

After initialization, JORMUNGANDR works in epochs. In each epoch, the allocator gets demands (i.e., required throughputs and tail latencies) from the data collected from last epoch, and produces a deployment that comprises new configurations for all GPUs. A deployment is valid if it satisfies demands: for each service, (i) the total throughput (from all instances) is greater than or equal to the required throughput in demands, and (ii) every instance's tail latency is smaller than what required by demands. JORMUNGANDR allows users to specify the latency percentile; by default, it is 99%-tile.

JORMUNGANDR then generates a transition plan that transfers the GPU cluster from the last deployment to the new one. Transition plan includes a series of instructions to create, delete instances, and repartition GPUs. When JORMUNGANDR executes this transition plan, end users (and the external world) should not observe performance violations (for example, a model offers a smaller throughput than its SLOs). After transition, the GPU cluster moves to a new epoch. Next, we introduce how JORMUNGANDR achieves the transition.

6 DEPLOYMENT TRANSITION

In practice, DNN service demands (gathered from the last epoch, §5) change from time to time. As a response, JORMUNGANDR periodically plans new deployments and switches GPU clusters from an old deployment to a new one. JORMUNGANDR's deployer is responsible for implementing these deployment transitions. A naive transition approach is to suspend all containers, repartition the underlying GPUs, and then resume all containers. Of course, for user-facing services like serving DNNs, this approach is unacceptable because all services will be unavailable during transitions. Instead, the deployer guarantees that the performance SLOs are not violated during the transition by gradually migrating containers.

The deployer runs a transition protocol named *exchange-and-compact*, which detangles (a) MIG requirement changes (e.g., the new deployment replaces a 2/7 instance with a 4/7 instance for vgg19) and (b) GPU repartitions (e.g., the new deployment needs a MIG=[3/7, 4/7] which does not exist in the old deployment). The exchange-and-compact has two phases: the first exchange phase focuses on the MIG instance changes,

and the second compact phase addresses GPU partition adjustments. The phase-wise design also provides the desired robust transition by enabling partial transition at a time.

Exchange phase. The exchange phase adjusts the existing MIG instances to the planned sizes by creating and deleting containers, without repartitioning the GPUs. In particular, the deployer calculates the differences in MIG instances between the old and new deployments for each model, denoted as Δ_i for the i_{th} model. For example, $\Delta_1 = [+4/7, -2/7]$ indicates that the first model requires a new 4/7 instance and needs to recycle an old 2/7 instance.

In each Δ_i , the deployer creates pairs of new and old MIG instances; each pair contains a new MIG instance (e.g., +4/7) and one or more old instances (e.g., -2/7), ensuring that the throughput of the new instance is greater than or equal to that of the old instances. The deployer then applies each pair by first creating a container with the new MIG instance (using extra GPUs if needed) and then deleting the old containers. After processing all pairs, the deployer creates new containers that could not be paired with old ones and deletes any remaining old containers. Throughout the process, each model is guaranteed to maintain its expected throughput¹ because new containers are always created before old ones are deleted.

Compact phase. With all MIG instances in the correct sizes, the deployer needs to compact containers to eliminate GPU fragmentation. The deployer achieves this by repartitioning GPUs and migrating containers.

At the beginning of the compact phase, the deployer creates a list of GPUs which are not fully occupied, denoted as S . If S is not empty, the deployer selects a $GPU_i \in S$ and identifies instances from other GPUs in S that, when combined, can saturate GPU_i .² The deployer then repartitions GPU_i and migrates containers to GPU_i . For example, if GPU_i is currently running only a 4/7 instance container, then the deployer can choose a 2/7 instance container and a 1/7 instance container from another GPU_j in S , repartitions GPU_i to “MIG=[4/7, 2/7, 1/7]”, and migrates containers from GPU_j to GPU_i . The running 4/7 instance container is not interrupted due to MIG’s partial repartition [8]. Finally, the deployer removes GPU_i from S because the GPU_i is now fully utilized. The deployer repeats this process until finishing the new deployment.

7 IMPLEMENTATION

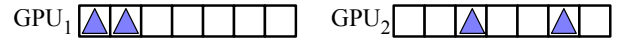
JORMUNGANDR is implemented in Python and Kubernetes (K8s). For the allocator, we implement the utility-first search (§4) in Python and NumPy. We implement the deployer based

on K8s [6]. JORMUNGANDR’s actions—instance creation, deletion, migration, and GPU partition—are wrappers of K8s operations and MIG operations via `nvidia-smi`. The allocator uses 3300 lines of Python code. The deployer uses 800 lines of Python, Bash, and Go code.

JORMUNGANDR uses a server-worker design where the allocator (§4) and the deployer (§6) run on a server node that manages a set of worker nodes. Periodically, the server sends actions (e.g., container creation and deletion) to workers. Workers then execute K8s operations accordingly; workers are machines that have GPUs and serve end-users’ requests.

During implementing JORMUNGANDR, we encounter multiple challenges due to limitations of Kubernetes (K8s) and NVIDIA tools (`nvidia-smi`). We elaborate on them below.

Pinpointing MIG locations. When allocating MIG instances, locations matter. For example, in the figure below, both GPU_1 and GPU_2 have two running 1/7 instances. GPU_1 allows allocating a 3/7 instance, but GPU_2 does not (see partition constraints in Figure 2).



However, K8s only allows a container to configure which MIG sizes it wants, without pinpointing the MIG location. The container configuration is ambiguous when there are multiple available MIG instances of the same size. To resolve this ambiguity, JORMUNGANDR deletes MIG instances whenever they’re empty and creates them only when creating a corresponding K8s container. So, when a container requires a MIG instance, there is only one available. This approach sacrifices opportunities of reusing MIG instances during transition for the container of the same size. The rationale is that MIG instance creation and deletions are cheap (<100ms, §8.3).

MIG instance rediscovery. In the compact phase, the deployer needs to repartition GPUs. But the device module of K8s (`nvidia-device-plugin`) lacks the ability to rediscover newly partitioned MIG instances. To make new partitions visible, the JORMUNGANDR reboots the module to trigger its GPU discovery procedure. Meanwhile, the deployer keeps actively pulling the changes to shorten the waiting time. It turns out that rediscoveries are the bottleneck of the deployment transition (§8.3). Each takes a few seconds to finish. In order to reduce the number of rediscoveries, we batch instance creations and deletions on the same GPU whenever we can. So, the discovery only needs to happen once for all the changes.

Fast resource release. In the exchange phase, the deployer frequently deletes K8s containers to release GPU resources. However, a graceful K8s deletion (`kubectl delete`) is extremely slow. It usually takes tens of seconds to finish, which is too expensive for JORMUNGANDR. We observe that containers serving DNN models are *stateless*, meaning that “inelegant” deletions do not affect system’s correctness. Thus,

¹The expected throughput is the throughput in either the old deployment or the new one, whichever is smaller.

²The selection order is not important because the MIG instances of the same size are identical, and the new deployment ensures that GPU configurations exist to accommodate all instances.

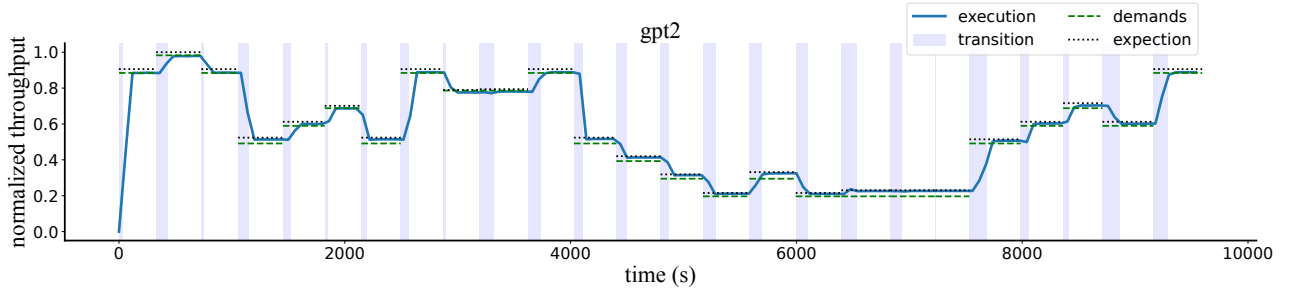


Figure 6: Normalized throughput of gpt2 during the 24-hour experiment. The solid line represents (“execution”) the observed model throughputs; the dashed line indicates (“demands”) the demanded throughputs that we collect from production trace; the dotted line represents (“expectation”) the expected throughputs calculated by the JORMUNGANDR’s profiling data. The shaded areas are when deployment transitions happen.

JORMUNGANDR implement a vicious but fast deletion: when deleting a K8s container, the deployer sends a SIGKILL signal to the container process which instantly releases the GPU resources, and then later recycles the container’s K8s metadata and clear the error messages. This approach accelerates the deletion by 30×, and the fast deletion finishes in less than 1s.

8 EXPERIMENTAL EVALUATION

In this section, we answer three main questions:

- How does JORMUNGANDR perform in practice?
- How does the utility-first search perform, versus baselines?
- How long does JORMUNGANDR take to complete a deployment transition, and how does that compare to baselines?

Workloads. We use two sets of workloads:

- Real-world workloads with five models: we collect 24 hour trace of five models from a production environment. The five DNN models are bert-base-cased, resnet101, albert-large-v2, resnet50 and gpt2. Note that we scale down models’ throughputs to fit into our testbed while preserving their throughputs’ relative amounts.
- Simulation workloads with 16 models: we further generate four workloads for 16 DNN models. In each workload, models’ demand throughputs are generated from either normal distributions (for two workloads) or lognormal distributions (for the other two workloads). Each workload requires few hundreds of GPUs. These simulation workloads serve as large workloads to test the algorithm performance.

Setup. We experiment with a four-machine GPU cluster that has eight A100 GPUs on Google Cloud Platform. Our testbed uses a2-highgpu-2g machines, each of which has 24 Intel Cascade Lake cores, 170GB memory, and two NVIDIA A100 40GB GPUs. The OS is Debian 4.19. We use PyTorch v1.12.1 and CUDA 11.6 for DNN inferences, and K8s v1.23.6.

8.1 Serving DNN models in practice

To experiment if JORMUNGANDR works as expected, we run a “24-hour” experiment, under the real-world workloads, on our cloud testbed. We set each JORMUNGANDR epoch to be 1 hour, so there will be 24 deployment transitions during the experiment. Because containers do not change within one epoch, we shorten the 1-hour normal running period (during which JORMUNGANDR is not involved) to 5 minutes to accelerate the experiment. Within each epoch (now 5min), we collect the throughputs and latencies of all models and record them as the execution trace. The K8s containers serve DNN models in a closed-loop to saturate the GPUs. Figure 6 shows the results for one model, gpt2. Other models show similar results.

The 24-hour experiment last about 2.5 hours (i.e., 9000 seconds). The white areas in Figure 6 indicate normal epochs with 5 minutes each; the shaded areas represent deployment transitions whose running time varies because of different deployments. Some adjacent deployments are similar while others are vastly different.

By comparing the demanded (the dashed line) and expected (the dotted line) throughputs in Figure 6, we see that JORMUNGANDR produce GPU configurations that provide just enough resources to satisfy the demands. Also, the actual throughputs (the solid line) overlap with the expected throughputs, which confirms our observation that DNN inferences have stable performance.

In each deployment transition (the shaded area in Figure 6), we see that the line of the actual throughput never drops below the smaller throughput between the old and new deployments. This demonstrates that the deployer can guarantee no performance violations during transitions. We further study transition procedures in section 8.3.

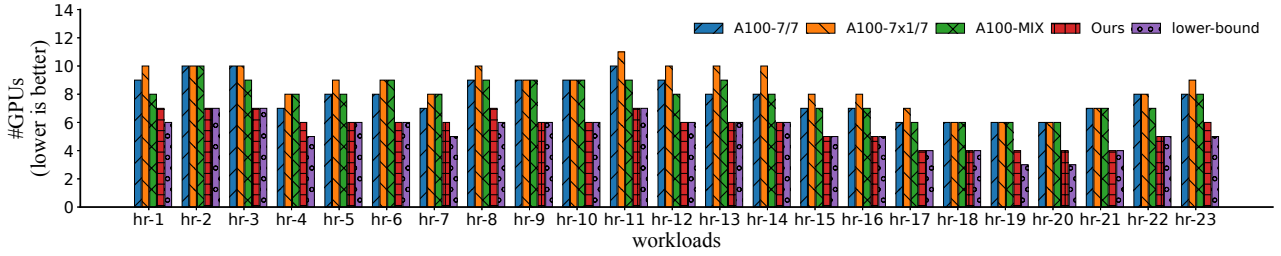


Figure 7: Number of GPUs used by algorithms for different real-world workloads. “A100-7/7”, “A100-7×1/7”, and “A100-MIX” are the three static allocation baselines; “Ours” represents the utility-first search; “lower-bound” is the minimal number of GPUs ignoring constraints.

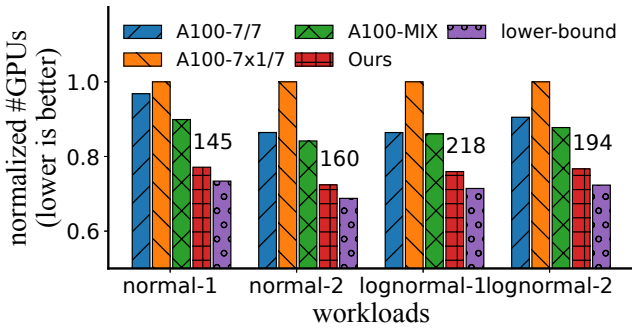


Figure 8: Number of GPUs used by algorithms for four simulation workloads. The numbers are normalized cluster-wise for each workload. The absolute numbers of JORMUNGANDR used GPUs are given.

8.2 Utility-first search performance

Baselines. We compare the utility-first search with two categories of baselines:

- *Static GPU partitions:* (1) A100-7×1/7, partitioning all GPUs into 1/7 instances, (2) A100-7/7, using A100 GPUs as-is, and (3) A100-MIX, using MIG=[4/7,2/7,1/7] for all A100s, and scheduling one model on one GPU. A100-7/7 uses GPU the traditional way (ignoring MIG). A100-MIX represents heterogeneous setup but does not serve different models on the same GPU.
- *Lower-bound estimation:* To see how well JORMUNGANDR performs compared to the optimal solution (which is computationally expensive to get), we calculate an approximate optimality—a lower bound of GPU usage by *ignoring* MIG’s hardware constraints. In particular, we assume that GPUs are continuous resources, for example, allowing to allocate 0.1 share of a 1/7 instance. Notice that the lower bound is impossible to achieve.

Static GPU partitions represent today’s MIG usage: developers partition GPUs first, and then statically assign each MIG

instances to containers. The lower-bound estimation is used to show how close utility-first search’s solutions to the optimal solutions. Lower-bound’s solutions are invalid in practice. We also discuss other possible baselines in section 10.

In this section, we study how utility-first search performs compared to baselines.

Real-world workloads. We run JORMUNGANDR and the baselines on the real-world workloads and count the number of GPUs they use. Figure 7 shows the results. JORMUNGANDR uses fewer GPUs than other baselines. Moreover, JORMUNGANDR is close to the optimal allocation—the largest gap to the lower bound is using one more GPU.

Simulation workloads. The workloads used are the four simulation workloads generated from normal and lognormal distributions, denoted as normal-1, normal-2, lognormal-1, and lognormal-2. We design the four workloads to use one to two hundreds of GPUs, representing a large-scale GPU cluster. There are 16 models included in all four workloads. Figure 8 shows the results. For all four workloads, JORMUNGANDR’s solutions are close to the lower-bound estimation: JORMUNGANDR uses < 6% more GPUs than the lower-bound, whereas other solutions all use > 20% more GPUs and A100-7×1/7 sometimes uses 40% more GPUs. Compared with static partition baselines, JORMUNGANDR can save up to 38% GPUs with an average saving of 22%.

8.3 Deployment transition

In the 24-hour experiment (§8.1), there are 24 deployment transitions. In this section, we want to take a closer look at how JORMUNGANDR applies these transitions.

End-to-end transition runtime. We measure the wall-clock time of JORMUNGANDR’s deployment transitions. Figure 9 shows the results. Most of the transitions finish within 2 minutes. In particular, the transition of “18->19” uses zero second because the deployments designed by the allocator are the same (the SLOs are similar for the two epochs).

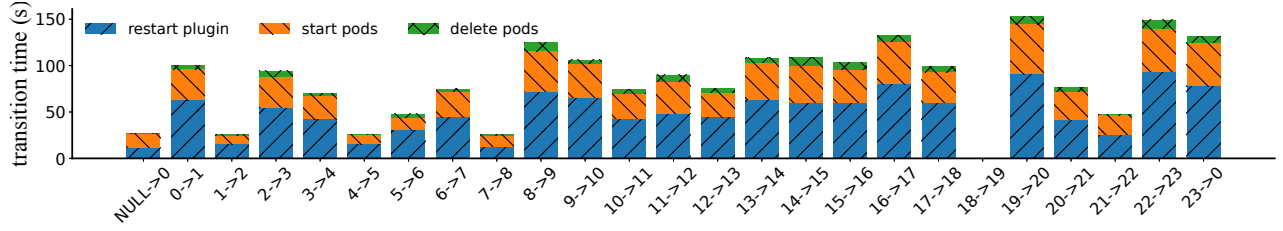


Figure 9: JORMUNGANDR’s transition time breakdown. The “restart plugin” represents the time JORMUNGANDR spent on rebooting K8s device module; the “start pod” and “delete pod” indicate the time spent on creating and deleting instances, respectively. The “X→Y” on x-axis represents the transition from the deployment of hour “X” to the hour “Y” in our 24-hour experiment.

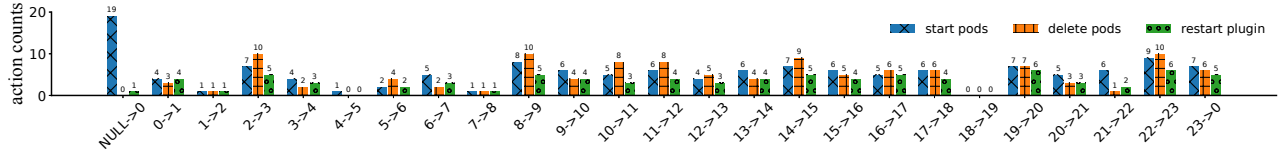


Figure 10: Operation counts during deployment transitions.

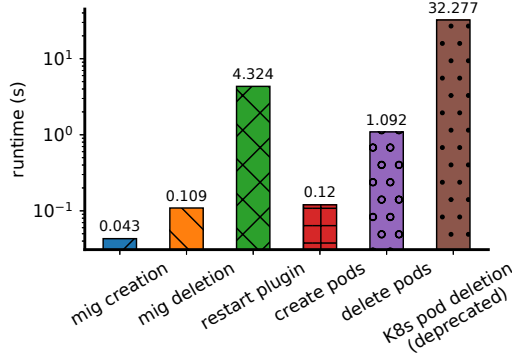


Figure 11: The average runtime of each transition operation. The y-axis is in log scale.

We further decompose each transition runtime into time spent on different components: rebooting the K8s device module, creating K8s pods, and deleting K8s pods. We find that rebooting K8s module is the slowest. Our near future work is to implement a native device recovery for K8s. By further analysis, it turns out that K8s’ creation is also expensive because K8s needs to bootstrap a container on a MIG instance. We believe DNN context switch techniques, like Pipeswitch [16], can significantly reduce this overhead.

A closer look at transitions. To understand the details of the transitions, we record the number of actions issued by JORMUNGANDR and summarize them in in Figure 10. Except for the first transition when the GPU cluster is empty, other transitions have balanced number of three operations. Compared with the runtime of these operations (Figure 9), we see that pod deletions are fast, confirming JORMUNGANDR’s fast resource release optimization (§7).

transition id	nos	ours	speedup
NULL→0	37.4	29.1 (29.1)	1.3× (1.3×)
3→4	78.9	70.4 (70.4)	1.1× (1.1×)
5→6	106.5	48.1 (48.1)	2.2× (2.2×)
22→23	215.9	151.2 (151.2)	1.4× (1.4×)
23→0	88.6	132.4 (66.4)	0.7× (1.3×)

Figure 12: Transition time in seconds for nos and exchange-and-compact (“ours”). For a fair comparison, we also list runtime of exchange-and-compact without compact in the parentheses.

Transition baseline: Nebuly Operating System. To the best of our knowledge, Nebuly Operating System (nos) [10] is the only system that manages MIG-enabled GPUs. It supports dynamic GPU partitions for MIG and MPS, as well as elastic resource quota management. We use nos as the baseline for the deployment transition. Because nos does not support “compact”, meaning that nos may use more GPUs than our system, we implement a version of exchange-and-compact that does not apply compaction. Figure 12 depicts the transition runtime for nos and JORMUNGANDR (w/ and w/o compaction). For the first three transitions (“NULL→0”, “3→4”, “5→6”), there is no need for compaction. JORMUNGANDR accelerate nos by 1.1–2.2×. For the transition “22→23”, there is a slight compaction, which doesn’t affect much about the performance. Finally, the transition “23→0” has a heavy compaction because deployment changes significantly. If considering compaction, JORMUNGANDR is slower than nos but uses fewer GPUs; if ignoring compaction (JORMUNGANDR will use the same number of GPUs as nos), JORMUNGANDR is 1.3× faster than nos.

Transition actions. During a transition, the deployer has five types of actions: MIG creation and deletion, K8s pod creation and deletion, and rebooting K8s device module. We measure

each of their runtime and show results in Figure 11. In this experiment, we run these actions 10 times in a synchronous manner—we issue an action and wait until it finishes.

9 RELATED WORK

GPU sharing. There is a line of work [20, 22, 28, 46, 49, 51, 54] studying GPU sharing for serving DNNs. They assume different setups. INFaaS [46] considers a model-less setup in which systems can substitute one model with similar models having the same functionality. Cocktail [28] allows an ensemble of models to serve user requests. GSLICE [22] uses MPS to share GPUs with an adaptive batching for models. JORMUNGANDR targets cloud native environment and is compatible to Kubernetes. Unlike prior GPU sharing, JORMUNGANDR considers a recent hardware feature, MIG. It turns out that sharing MIG-enabled GPUs is a harder problem (§3.3).

Resource allocation. Resource allocation [14, 31, 42] is a broach topic that have many variants. Allocating MIG-enabled GPUs is a new problem (§3.3) that combines three characteristics: (a) instance performance does not grow linearly with additional resources (§3.1), (b) MIG-enabled GPUs follow restricted reconfiguration rules (§2), and (c) MIG supports partial reconfiguration. Traditional resource allocation methods do not address all these characteristics. For example, CPU allocators usually assume cores are identical, hence violates (a). Memory allocators assume a continuous set of pages can be allocated together, contradicting (b). Reconfigurable Machine Tools (RMTs) [35] allows rearranging machines, but must work in a fixed-sized reconfigurable unit, ignoring (c).

A recent work *Partitioned Optimization Problems (POP)* [39] addresses difficult resource allocation problems by randomly splitting the original problem into smaller ones, solving smaller problems separately, and coalescing the results. POP requires the targeted problem satisfying four conditions, one of which needs the resources to be “fungible” [39, §2, Condition 3]. But in our problem, resources are not fungible because a unit of resource contributes differently in different sized instances.

Scheduling problem. Scheduling problems [45] have been well-studied by multiple communities. Some of them are related to our problem. The problem UPM [45] (*Unrelated Parallel Machine Scheduling Problem*) tackles different sized instances have different performance, but it requires a fixed set of machines and does not capture MIG’s reconfigurability. As mentioned, problems with RMTs [35] (*Reconfigurable Machine Tools*) address GPU’s reconfigurability. Examples are FJSSP-CDST [37] and permutation flow shop plus RMTs [15]. But they do not support partial reconfiguration because RMTs have fixed-sized reconfigurable units. The problem DCSP [25, 34] (*Discrete-Continuous Scheduling Problem*) handles partial reconfiguration, but it requires the

resources to be continuous (like power). But GPU has discrete resources (namely, GPU SMs and memory banks) and their allocation is restricted by MIG hardware constraints (§2).

Partial reconfiguration. Similar to MIG-enabled GPUs, FPGA also supports partial reconfiguration [3]. A classic reconfigurable device model is the 2D resource model [48, 50] which abstracts a job with two constraints, *width* and *height*, representing the runtime and spatial constraints. The 2D model targets a similar but different problem: a job in the model has a fixed width and height, which in our context means that a service can use one fixed-sized GPU instance.

DNN serving systems. Traditional DNN serving systems—for example, Tensorflow-serving [40], TorchServe [2], Clipper [21]—mainly focus on optimizing the performance for a single service instance, whereas JORMUNGANDR works on a layer below their abstraction: JORMUNGANDR targets planning resource allocation hence is complementary to them; these frameworks can run in containers created by JORMUNGANDR.

Another thread of DNN serving systems [17, 24, 27, 46, 47] aims at different workloads or specific DNN models. Though, conceptually, JORMUNGANDR are complementary to these systems and can run beneath them, it requires further research to unleash the full potential of combining both systems. For example, BatchMaker [24] improves RNN’s inference performance by cellular batching. Similarly, Nexus [47] accelerates DNN serving by batching requests partially across different DNN models. Because the acceleration of these systems depends on how much portion of requests can be batched, serving performance varies with workloads. This is a challenge for JORMUNGANDR as our algorithms require stable performance as inputs. Likewise, Clockwork [27] requires the full control over the execution stack to achieve predictable inference latency, whose scheduling decisions may conflict with JORMUNGANDR’s.

DNN training resource allocation. Allocating resources for DNN training, especially distributed training, has been intensively studied [19, 26, 36, 43, 44, 52, 55]. JORMUNGANDR only targets DNN serving because MIG’s GPU instances cannot use NVLink or NVSwitch to communicate [5]. This prohibits MIG from being used for multi-GPU training. If NVIDIA enables NVLink and NVSwitch for MIG, then it is an interesting and open problem of how to use MIG for training.

10 DISCUSSION

Assumptions. JORMUNGANDR operates under several assumptions. First, we assume there are enough GPUs to meet the Service Level Objectives (SLOs). If this assumption fails, the utility-first search will terminate without producing a valid deployment (Figure 4, line 19), at which point JORMUNGANDR will alert the user without altering the current deployment. A

similar outcome occurs if the transition (§6) lacks sufficient GPUs to complete. Second, we assume that out-of-memory errors will be detected during profiling. If a model runs out of memory with a given GPU configuration, we expect to identify this issue during profiling. Third, we assume that performance profiling is accurate, as observed by Gujarati et al. [27] and corroborated by our experiments. If this fails, then JORMUNGANDR may violate SLOs.

JORMUNGANDR as a resource allocator. As mentioned (§1), JORMUNGANDR functions as an allocator, distinct from but complementary to schedulers and load balancers. JORMUNGANDR relies on a load balancer to dispatch requests to containers and address request fluctuation. What JORMUNGANDR guarantees, assuming the above assumptions, is that when the instant requested throughput does not exceed the SLOs, the serving latency will meet the SLOs. For example, if the request rate is lower than the expected throughput, the serving software will batch fewer requests, resulting in better turnaround time. Conversely, if there is a spike in requests exceeding the SLO-defined throughput, the load balancer—such as K8S Autoscaler [13]—will need to temporarily allocate additional containers and GPUs.

Tuning JORMUNGANDR. JORMUNGANDR involves several trade-offs which can be adjusted by tuning its parameters. For instance, it uses {1, 2, 4} as MPS options and batch sizes from {1, 2, 4, 8, 16, 32}. These MPS and batch size options can be added or removed, according to the specific use cases. In general, removing options decreases the likelihood of achieving optimal GPU utilization but increases the algorithm’s speed.

When to trigger transitions. JORMUNGANDR initiates deployment transitions when adding, removing, or updating models, as well as periodically (by default, every hour). The trade-off lies in the frequency of these transitions: frequent transitions optimize immediate GPU utilization but incur higher transition costs, while infrequent transitions reduce costs but may not adequately meet real-time user demands. Beyond transition frequency, using continuous allocation (§4.2) can also reduce transition costs, albeit at the expense of deploying suboptimal GPU configurations.

Deploying JORMUNGANDR on MIG-disabled GPUs. MIG is only available on a limited number of GPU types [8], including NVIDIA A30, A100, H100, and H200. However, many other GPU types do not support this hardware feature. Despite this, JORMUNGANDR remains functional on clusters with both MIG-enabled and MIG-disabled GPUs by omitting MIG options from GPU configurations that do not support MIG. For example, an NVIDIA A5000 would have a configuration of $\langle \text{MIG}=[], \text{MPS}=[1,2], \text{batch}=[8,16] \rangle$. The utility-first search (Figure 4, §4) remains unchanged, except that the “*conf_ser*” (Figure 4, line 3) include the mentioned MIG-disabled GPU

configurations. In a GPU cluster without any MIG devices, JORMUNGANDR may not perform as efficient, as it does not fully explore the potential search space, such as running different-sized MPS on the same GPU. Existing systems [20, 30] could perform better in this scenario.

Other solutions to RMAP. Finding solutions to the Reconfigurable Machine Allocation Problem (§3.3) is challenging, primarily because we are among the first to study this problem in a practical context. Recent theoretical work [18] has highlighted the inherent difficulty of the problem, showing that so far, the best achievable result is an asymptotic $(2+\epsilon)$ -approximation in polynomial time. However, despite being polynomial, this algorithm is not fast enough for practical use. We have also explored several alternative approaches, including exhaustive search, using heuristics to configuring MIG, formulating the problem as a linear programming or SMT problem, and using a solver like Z3 to solve it. Unfortunately, all these methods are either too slow or produce results inferior to our current solution. In addition, the past scheduling systems are orthogonal to MIGs hence hard to compare apples-to-apples with JORMUNGANDR.

Number of MIG instances. Currently, JORMUNGANDR is effective because the number of possible MIG configurations remains manageable: with 7 MIG slices, there are 19 distinct MIG partitions. However, as the number of MIG slices increases, the number of possible partitions will grow exponentially. If the number of MIG slices doubles or triples, the current approach will become intractable. At that point, we will need new strategies to conduct more efficient searches.

Profiling versus performance estimation. JORMUNGANDR uses profiling (§4.1) to gather performance data for different models across various GPU configurations. Profiling a model on all configurations—encompassing different MIG sizes, MPS numbers, and batch sizes—typically takes tens of minutes. To accelerate this process, one can estimate or predict model performance through simulation. However, in our targeted setup, where companies often use a model for a long period of time, accurate profiling is worth the time.

11 CONCLUSION

This paper studies how to use a new hardware feature MIG for DNN serving. Because of MIG’s characteristics, we find that the problem of allocating MIG-enabled GPUs is a new abstract problem, which we define as the Reconfigurable Machine Allocation Problem. To solve this problem, we introduce a utility-first search that produces near-optimal solutions in practice. Further, we design and implement a system, JORMUNGANDR, that produces valid deployments and implement them transparently to GPUs. Evaluation shows that JORMUNGANDR significantly outperforms baselines that statically partition A100s.

REFERENCES

- [1] NVIDIA Triton.
<https://developer.nvidia.com/nvidia-triton-inference-server>.
- [2] TorchServe. <https://pytorch.org/serve/>.
- [3] Vivado Design Suite User Guide Partial Reconfiguration.
https://www.xilinx.com/support/documentation/sw_manuals/xilinx2018_1/ug909-vivado-partial-reconfiguration.pdf, 2018.
- [4] Cutting stock problem.
https://en.wikipedia.org/wiki/Cutting_stock_problem, 2021.
- [5] Fabric Manager for NVIDIA NVSwitch Systems.
<https://docs.nvidia.com/datacenter/tesla/pdf/fabric-manager-user-guide.pdf>, 2021.
- [6] Kubernetes Controllers.
<https://kubernetes.io/docs/concepts/architecture/controller/>, 2021.
- [7] Minimizing Deep Learning Inference Latency with NVIDIA Multi-Instance GPU. <https://developer.nvidia.com/blog/minimizing-dl-inference-latency-with-mig/>, 2021.
- [8] NVIDIA Multi-Instance GPU User Guide. https://docs.nvidia.com/datacenter/tesla/pdf/NVIDIA_MIG_User_Guide.pdf, 2021.
- [9] AWS Deep Learning Containers.
<https://aws.amazon.com/machine-learning/containers/>, 2023.
- [10] Nebuly Operating System (nos). <https://github.com/nebuly-ai/nos>, 2023.
- [11] NVIDIA Multi-Process Service.
<https://docs.nvidia.com/deploy/mps/index.html>, 2023.
- [12] Use Deep Learning VM Images and Deep Learning Containers with Vertex AI.
<https://cloud.google.com/vertex-ai/docs/general/deep-learning>, 2023.
- [13] Kubernetes Autoscaler. <https://github.com/kubernetes/autoscaler>, 2024.
- [14] A. Ahmad, S. Ahmad, M. H. Rehmani, and N. U. Hassan. A survey on radio resource allocation in cognitive radio sensor networks. *IEEE Communications Surveys & Tutorials*, 17(2):888–917, 2015.
- [15] A. Azab and B. Naderi. Modelling the problem of production scheduling for reconfigurable manufacturing systems. *Procedia CIRP*, 33:76–80, 2015.
- [16] Z. Bai, Z. Zhang, Y. Zhu, and X. Jin. Pipeswitch: Fast pipelined context switching for deep learning applications. In *Proc. OSDI*, 2020.
- [17] A. Balasubramanian, A. Kumar, Y. Liu, H. Cao, S. Venkataraman, and A. Akella. Accelerating deep learning inference via learned caches. *arXiv preprint arXiv:2101.07344*, 2021.
- [18] M. Casey, R. Rajaraman, D. Stalf, and C. Tan. Scheduling splittable jobs on configurable machines. *arXiv preprint arXiv:2312.05416*, 2023.
- [19] S. Chaudhary, R. Ramjee, M. Sivathanu, N. Kwatra, and S. Viswanatha. Balancing efficiency and fairness in heterogeneous gpu clusters for deep learning. In *Proceedings of the Fifteenth European Conference on Computer Systems*, 2020.
- [20] S. Choi, S. Lee, Y. Kim, J. Park, Y. Kwon, and J. Huh. Serving heterogeneous machine learning models on {Multi-GPU} servers with {Spatio-Temporal} sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 199–216, 2022.
- [21] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica. Clipper: A low-latency online prediction serving system. In *Proc. NSDI*, 2017.
- [22] A. Dhakal, S. G. Kulkarni, and K. Ramakrishnan. Gslice: controlled spatial sharing of gpus for a scalable inference platform. In *Proceedings of the 11th ACM Symposium on Cloud Computing*, pages 492–506, 2020.
- [23] J. Fried, Z. Ruan, A. Ousterhout, and A. Belay. Caladan: Mitigating interference at microsecond timescales. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 281–297, 2020.
- [24] P. Gao, L. Yu, Y. Wu, and J. Li. Low latency rnn inference with cellular batching. In *Proc. EuroSys*, 2018.
- [25] M. Górczyca, A. Janiak, and W. Janiak. The discrete part of the discrete-continuous scheduling problems-new properties. *IFAC Proceedings Volumes*, 42(13):244–249, 2009.
- [26] J. Gu, M. Chowdhury, K. G. Shin, Y. Zhu, M. Jeon, J. Qian, H. Liu, and C. Guo. Tiresias: A {GPU} cluster manager for distributed deep learning. In *Proc. NSDI*, 2019.
- [27] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace. Serving dnns like clockwork: Performance predictability from the bottom up. In *Proc. OSDI*, 2020.
- [28] J. R. Gunasekaran, C. S. Mishra, P. Thinakaran, B. Sharma, M. T. Kandemir, and C. R. Das. Cocktail: A multidimensional optimization for model serving in cloud. In *USENIX NSDI*, pages 1041–1057, 2022.
- [29] M. Han, H. Zhang, R. Chen, and H. Chen. Microsecond-scale preemption for concurrent {GPU-accelerated}{DNN} inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 539–558, 2022.
- [30] Z. Han, R. Zhou, C. Xu, Y. Zeng, and R. Zhang. Inss: An intelligent scheduling orchestrator for multi-gpu inference with spatio-temporal sharing. *IEEE Transactions on Parallel and Distributed Systems*, 2024.
- [31] J. G. Herrera and J. F. Botero. Resource allocation in nfv: A comprehensive survey. *IEEE Transactions on Network and Service Management*, 13(3):518–532, 2016.
- [32] P. Jain, X. Mo, A. Jain, H. Subbaraj, R. S. Durrani, A. Tumanov, J. Gonzalez, and I. Stoica. Dynamic space-time scheduling for gpu inference. *arXiv preprint arXiv:1901.00041*, 2018.
- [33] K. Jansen, A. Lassota, and M. Maack. Approximation algorithms for scheduling with class constraints. In *Proceedings of the 32nd ACM Symposium on Parallelism in Algorithms and Architectures*, pages 349–357, 2020.
- [34] J. Józefowska, M. Mika, R. Różycki, G. Waligóra, and J. Węglarz. Solving discrete-continuous scheduling problems by tabu search. In *4th Metaheuristics International Conference MIC*, pages 16–20, 2001.
- [35] R. G. Landers, B.-K. Min, and Y. Koren. Reconfigurable machine tools. *CIRP Annals*, 50(1):269–274, 2001.
- [36] K. Mahajan, A. Balasubramanian, A. Singhvi, S. Venkataraman, A. Akella, A. Phanishayee, and S. Chawla. Themis: Fair and efficient {GPU} cluster scheduling. In *Proc. NSDI*, 2020.
- [37] M. Mahmoodjanloo, R. Tavakkoli-Moghaddam, A. Baboli, and A. Bozorgi-Amiri. Flexible job shop scheduling problem with reconfigurable machine tools: An improved differential evolution algorithm. *Applied Soft Computing*, 94:106416, 2020.
- [38] S. McClure, A. Ousterhout, S. Shenker, and S. Ratnasamy. Efficient scheduling policies for {Microsecond-Scale} tasks. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 1–18, 2022.
- [39] D. Narayanan, F. Kazhamiaka, F. Abuzaid, P. Kraft, A. Agrawal, S. Kandula, S. Boyd, and M. Zaharia. Solving large-scale granular resource allocation problems efficiently with pop. In *Proc. SOSP*, 2021.
- [40] C. Olston, N. Fiedel, K. Gorovoy, J. Harmsen, L. Lao, F. Li, V. Rajashekhar, S. Ramesh, and J. Soyke. Tensorflow-serving: Flexible, high-performance ml serving. *arXiv preprint arXiv:1712.06139*, 2017.
- [41] A. Ousterhout, J. Fried, J. Behrens, A. Belay, and H. Balakrishnan. Shenango: Achieving high {CPU} efficiency for latency-sensitive datacenter workloads. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 361–378, 2019.
- [42] M. Patriksson. A survey on the continuous nonlinear resource allocation problem. *European Journal of Operational Research*, 185(1):1–46, 2008.

- [43] Y. Peng, Y. Bao, Y. Chen, C. Wu, and C. Guo. Optimus: an efficient dynamic resource scheduler for deep learning clusters. In *Proc. EuroSys*, 2018.
- [44] Y. Peng, Y. Zhu, Y. Chen, Y. Bao, B. Yi, C. Lan, C. Wu, and C. Guo. A generic communication scheduler for distributed dnn training acceleration. In *Proc. SOSP*, 2019.
- [45] M. Pinedo. *Scheduling*, volume 29. Springer, 2012.
- [46] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis. Infaas: A model-less inference serving system. *arXiv preprint arXiv:1905.13348*, 2019.
- [47] H. Shen, L. Chen, Y. Jin, L. Zhao, B. Kong, M. Philipose, A. Krishnamurthy, and R. Sundaram. Nexus: a gpu cluster engine for accelerating dnn-based video analysis. In *Proc. SOSP*, 2019.
- [48] C. Steiger, H. Walder, and M. Platzner. Operating systems for reconfigurable embedded platforms: Online scheduling of real-time tasks. *IEEE Transactions on computers*, 53(11):1393–1407, 2004.
- [49] L. Wang, L. Yang, Y. Yu, W. Wang, B. Li, X. Sun, J. He, and L. Zhang. Morphling: Fast, near-optimal auto-configuration for cloud-native model serving. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 639–653, 2021.
- [50] Z. Wang, Q. Tang, B. Guo, J.-B. Wei, and L. Wang. Resource partitioning and application scheduling with module merging on dynamically and partially reconfigurable fpgas. *Electronics*, 9(9):1461, 2020.
- [51] X. Wu, H. Xu, and Y. Wang. Irina: Accelerating dnn inference with efficient online scheduling. In *4th Asia-Pacific Workshop on Networking*, pages 36–43, 2020.
- [52] W. Xiao, R. Bhardwaj, R. Ramjee, M. Sivathanu, N. Kwatra, Z. Han, P. Patel, X. Peng, H. Zhao, Q. Zhang, et al. Gandiva: Introspective cluster scheduling for deep learning. In *Proc. OSDI*, 2018.
- [53] W. Xing and J. Zhang. Parallel machine scheduling with splitting jobs. *Discrete Applied Mathematics*, 103(1-3):259–269, 2000.
- [54] H. Zhang, Y. Tang, A. Khandelwal, and I. Stoica. {SHEPHERD}: Serving {DNNs} in the wild. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 787–808, 2023.
- [55] H. Zhao, Z. Han, Z. Yang, Q. Zhang, F. Yang, L. Zhou, M. Yang, F. C. Lau, Y. Wang, Y. Xiong, et al. Hived: Sharing a {GPU} cluster for deep learning with guarantees. In *Proc. OSDI*, 2020.